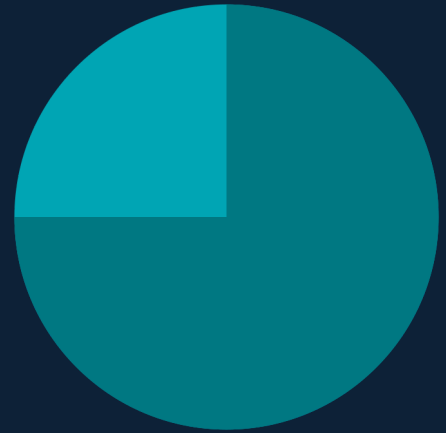




COMPTE RENDU

TP Spring MVC : Application Web JEE

Gestion des Produits

Pr. Mohamed CHERRADI
J2EE, Java EE (JEE)
TDIA2-S4 / 2025-26

Hiba El ouazi

1

Introduction

Objectif et contexte du TP

Ce compte rendu présente la réalisation du **TP Spring MVC : Application Gestion des Produits**, dans le cadre du module **J2EE (Jakarta EE)**. L'objectif est de faire évoluer l'architecture **MVC2** vers **Spring MVC**, le framework Java le plus utilisé en entreprise, tout en conservant la persistance **Hibernate + MySQL**, l'**authentification** par session et la **gestion des rôles** (ADMIN/USER).

Spring MVC DispatcherServlet + @Controller + @Autowired	Hibernate ORM Persistance MySQL via Session + HQL	Sécurité AuthFilter + RoleFilter + Session HTTP
--	--	--

Par rapport au TP MVC2 précédent, Spring MVC apporte une **injection de dépendances automatique** (@Autowired), un **routing déclaratif** (@GetMapping, @PostMapping), et une **configuration XML** (spring-beans.xml) pour les beans DAO et Service.

2

Architecture du Projet

Flux de navigation et composants

L'application suit une architecture **Spring MVC 3-tiers**. Le **DispatcherServlet Spring** (configuré dans web.xml) intercepte toutes les requêtes et les route vers le **ProduitController** selon les annotations @GetMapping/@PostMapping. Les beans DAO et Service sont injectés via **spring-beans.xml**.

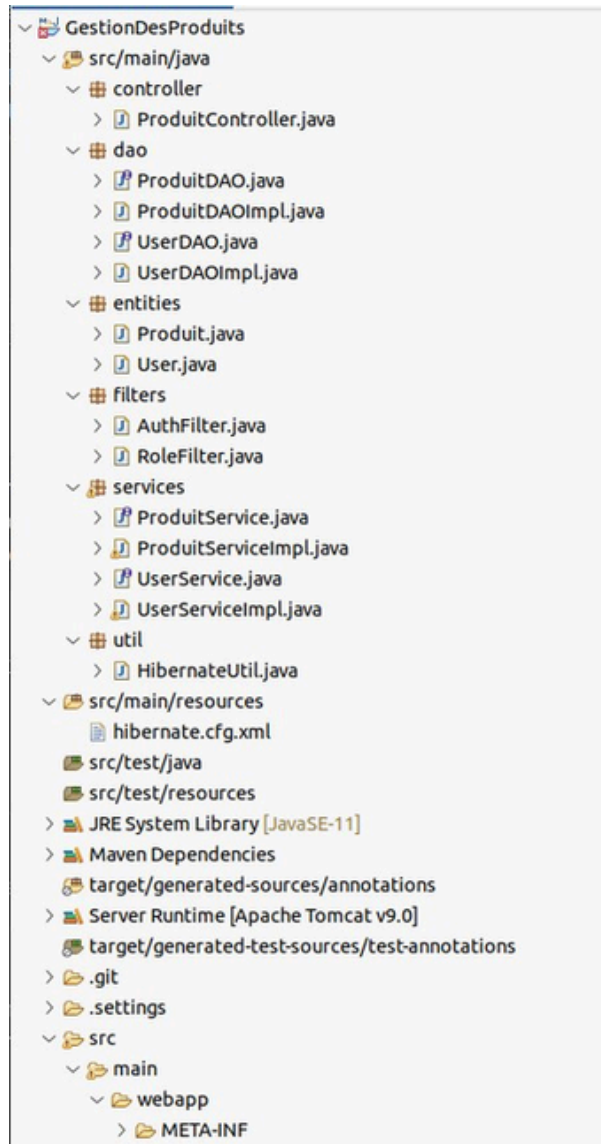
URL	Controller / Handler	Rôle	Méthode
/login	ProduitController.loginForm()	Afficher formulaire de connexion	GET
/login	ProduitController.loginSubmit()	Authentification + création session	POST
/logout	ProduitController.logout()	Destruction de la session	GET
/produits	ProduitController.listeProduits()	Afficher liste complète (protégée)	GET
/produits/search	ProduitController.searchProduit()	Recherche par ID ou mot-clé	GET
/produits/add	ProduitController.addProduit()	Ajouter un produit (ADMIN)	POST
/produits/edit	ProduitController.editProduit()	Charger formulaire modification (ADMIN)	GET
/produits/update	ProduitController.updateProduit()	Mettre à jour un produit (ADMIN)	POST
/produits/delete	ProduitController.deleteProduit()	Supprimer un produit (ADMIN)	GET

Flux de navigation :

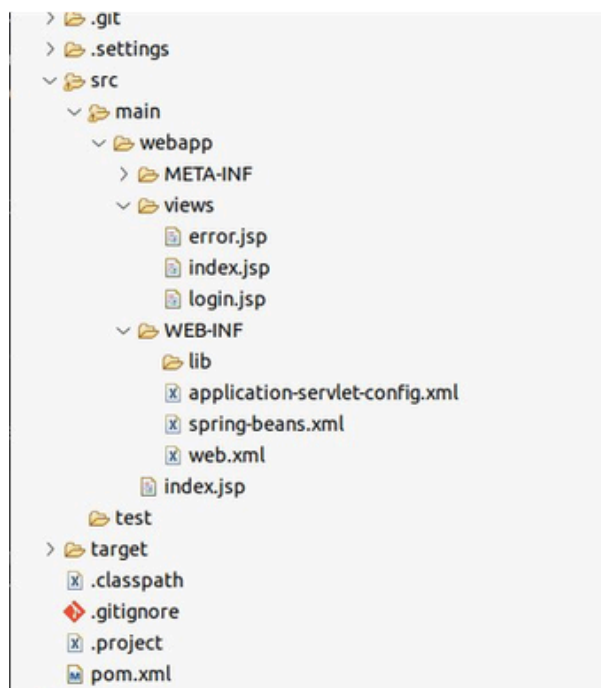
Flux	Description
Non connecté → /produits	AuthFilter détecte pas de session → redirect /login
POST /login	ProduitController authentifie → session.setAttribute('userConnecte') → redirect /produits
POST /produits/add	RoleFilter vérifie ADMIN → ProduitController → ProduitService → Hibernate INSERT MySQL
GET /logout	ProduitController invalide la session → redirect /login

3 Structure du Projet

Le projet **GestionDesProduits** est un **Maven Project** Eclipse avec Tomcat 9. Il utilise **Spring MVC 5.3** (javax.servlet, compatible Tomcat 9) et **Hibernate 5.6** pour la persistance MySQL. Les JSPs sont dans **src/main/webapp/views/** et les classes Java organisées par couche (controller, dao, entities, services, filters, util).



• Structure du projet GestionDesProduits partie 1



- Structure du projet GestionDesProduits partie 2

Fichier	Emplacement	Rôle
<i>ProduitController.java</i>	src/main/java/controller/	@Controller Spring : CRUD + Auth (login/logout)
<i>ProduitDAO / ProduitDAOImpl</i>	src/main/java/dao/	Interface DAO + implémentation Hibernate (session.save/update/delete)
<i>UserDAO / UserDAOImpl</i>	src/main/java/dao/	Authentification via HQL (findByUsernameAndPassword)
<i>Produit.java / User.java</i>	src/main/java/entities/	Entités JPA (@Entity, @Table, @Id) ; mappées MySQL
<i>ProduitService / Impl</i>	src/main/java/services/	Logique métier + validation (nom vide, prix négatif)
<i>UserService / Impl</i>	src/main/java/services/	Authentification + init utilisateurs en BDD au démarrage
<i>AuthFilter.java</i>	src/main/java/filters/	Filtre session HTTP ; redirige vers /login si non connecté
<i>RoleFilter.java</i>	src/main/java/filters/	Filtre rôle ; réserve add/edit/update/delete aux ADMIN
<i>HibernateUtil.java</i>	src/main/java/util/	Singleton SessionFactory Hibernate (une seule instance)
<i>hibernate.cfg.xml</i>	src/main/resources/	Config Hibernate : URL MySQL, dialect, hbm2ddl.auto=update
<i>login.jsp / index.jsp / error.jsp</i>	src/main/webapp/views/	Vues JSP + JSTL ; même style que MVC2 précédent
<i>web.xml</i>	src/main/webapp/WEB-INF/	DispatcherServlet Spring + filtres AuthFilter + RoleFilter
<i>spring-beans.xml</i>	src/main/webapp/WEB-INF/	Beans DAO + Service injectés par Spring IoC
<i>application-servlet-config.xml</i>	src/main/webapp/WEB-INF/	mvc:annotation-driven + component-scan + ViewResolver
<i>pom.xml</i>	racine/	Dépendances Maven : Spring MVC 5.3, Hibernate 5.6, MySQL

4 Implémentation du Code

L'implémentation repose sur trois configurations XML Spring et des classes Java annotées. Le **spring-beans.xml** déclare les beans DAO et Service avec injection de dépendances. Le **application-servlet-config.xml** active les annotations MVC et le ViewResolver. Le **web.xml** déclare le DispatcherServlet Spring et les deux filtres de sécurité.

4.1 Entités JPA — Produit.java et User.java

Les entités utilisent les annotations **javax.persistence** (compatible Hibernate 5.x + Tomcat 9). **@Entity** indique à Hibernate que la classe est mappée sur une table MySQL. **@GeneratedValue(strategy=IDENTITY)** configure l'auto-incrémentation. Hibernate crée automatiquement les tables au démarrage grâce à **hbm2ddl.auto=update**.

Annotation	Rôle	Équivalent SQL
@Entity	Déclare la classe comme entité JPA	CREATE TABLE ...
@Table (name='produits')	Spécifie le nom de la table	TABLE produits
@Id	Clé primaire de l'entité	PRIMARY KEY
@GeneratedValue (IDENTITY)	Auto-incrémentation de l'ID	AUTO_INCREMENT
@Column (nullable=false)	Colonne non nulle	NOT NULL

4.2 HibernateUtil.java — Singleton SessionFactory

La **SessionFactory** est coûteuse à créer (lecture hibernate.cfg.xml, connexion MySQL, mapping des entités). On l'instancie **une seule fois** au chargement de la classe via un bloc **static{}**. Toutes les opérations DAO ouvrent une Session légère depuis cette factory et la ferment dans un bloc **finally** pour éviter les fuites mémoire.

Méthode Hibernate	Action SQL générée	Quand l'utiliser
session.save(obj)	INSERT INTO produits ...	Ajouter un NOUVEL objet
session.get(Class, id)	SELECT * WHERE id = ?	Récupérer par ID
session.update(obj)	UPDATE produits SET ...	Modifier un objet existant
session.delete(obj)	DELETE FROM produits WHERE id=?	Supprimer un objet
session.createQuery(HQL)	SELECT ... (traduit en SQL)	Requête personnalisée HQL

4.3 ProduitController.java — Le Controller Spring MVC

Le **ProduitController** est annoté **@Controller** ; Spring le détecte automatiquement via **component-scan**. Les services sont injectés par **@Autowired** : Spring trouve le bean correspondant dans spring-beans.xml et l'injecte sans instantiation manuelle.

Chaque méthode handler utilise **@GetMapping** ou **@PostMapping** pour mapper une URL. La méthode reçoit un **Model** pour passer les données à la JSP, et retourne le **nom logique de la vue** (ex: 'index') que le **ViewResolver** transforme en /views/index.jsp.

Annotation	Rôle dans Spring MVC
------------	----------------------

<code>@Controller</code>	Marque la classe comme Controller Spring MVC détectable par component-scan
<code>@Autowired</code>	Spring injecte automatiquement produitService et userService depuis spring-beans.xml
<code>@GetMapping('/produits')</code>	Mappe GET /produits → méthode listeProduits(Model model)
<code>@PostMapping('/produits/add')</code>	Mappe POST /produits/add → méthode addProduit(@RequestParam...) Récupère un paramètre de l'URL ou du formulaire (nom, prix, id...)
<code>@RequestParam</code>	Passe des messages flash après un redirect (pattern PRG)
<code>RedirectAttributes</code>	
<code>return 'redirect:/produits'</code>	Redirige vers /produits après opération POST (pattern PRG)
<code>return 'index'</code>	Forward vers /views/index.jsp via le ViewResolver

4.4 spring-beans.xml : Injection de Dépendances XML

Le **spring-beans.xml** est le cœur de l'**IoC Container** Spring. Il déclare les beans DAO et Service et configure leurs dépendances. La balise `<property name='dao' ref='produitDAO'/>` déclenche l'appel de `setDao(produitDAO)` automatiquement par Spring. Le `init-method='initDefaultUsers'` sur `userService` insère les utilisateurs par défaut en base MySQL au démarrage de l'application.

Bean ID	Classe	Injection
<code>produitDAO</code>	<code>dao.ProduitDAOImpl</code>	Aucune (implémente ProduitDAO)
<code>userDAO</code>	<code>dao.UserDAOImpl</code>	Aucune (implémente UserDAO)
<code>produitService</code>	<code>services.ProduitServiceImpl</code>	<code>dao</code> → ref produitDAO (via setDao())
<code>userService</code>	<code>services.UserServiceImpl</code>	<code>userDAO</code> → ref userDAO + init-method=initDefaultUsers

4.5 AuthFilter.java et RoleFilter.java

Les deux filtres (javax.servlet) interceptent les requêtes **avant** que Spring MVC les traite. **AuthFilter** vérifie la session HTTP sur toutes les URLs (/*) et redirige vers /login si absente. **RoleFilter** est appliqué uniquement sur les URLs d'action sensibles (/produits/add, /produits/edit, /produits/update, /produits/delete) et vérifie que l'utilisateur possède le rôle **ADMIN**. Si ce n'est pas le cas, il redirige vers /produits avec un message d'erreur.

Filtre	URL Pattern	Condition	Action si KO
<code>AuthFilter</code>	<code>/*</code>	<code>session.getAttribute('userConnecte') != null ?</code>	redirect /login
<code>RoleFilter</code>	<code>/produits/add</code> <code>/edit /update</code> <code>/delete</code>	<code>user.getRole() == 'ADMIN' ?</code>	redirect /produits?messageErreur=...

5

Captures d'Écran de l'Application

Résultats obtenus

L'application est déployée sur **Apache Tomcat 9.0** et accessible à **http://localhost:8084/GestionDesProduitsSPRING/**. Les fonctionnalités sont identiques au TP MVC2 précédent : connexion, CRUD produits, recherche, gestion des rôles et déconnexion. La différence majeure est l'utilisation de **Spring MVC** côté Controller et la **persistance MySQL via Hibernate**.

Page de Connexion

Gestion des Produits

Nom d'utilisateur ou mot de passe incorrect.

NOM D'UTILISATEUR

slk

MOT DE PASSE

Entrez votre mot de passe

Se connecter

- Formulaire de connexion (login.jsp) ; Spring MVC POST /login

Tableau de Bord : Profil ADMIN

Après connexion ADMIN, l'interface complète est accessible. Le formulaire d'ajout à gauche envoie un POST /produits/add traité par **@PostMapping** dans le Controller Spring.

The screenshot shows the Admin interface with a blue header bar containing the URL '1081/GestionDesProduitsMVC1/listProduits' and a user profile 'admin' with a 'Déconnexion' button. The main content area is divided into two sections:

Ajouter un produit

Form fields:

- Nom du produit *
- Description *
- Prix (MAD) *

Ajouter le produit

Liste des produits

Recherche par ID... Nom ou description... Chercher Reset

5 produit(s)

ID	NOM	DESCRIPTION	PRIX	ACTIONS
#1	PC Portable Dell	Dell Inspiron i7, 16Go RAM	8500.0 MAD	Modifier Supprimer
#2	PC Portable HP	HP Pavilion i5, 8Go RAM	6200.0 MAD	Modifier Supprimer
#3	Smartphone Samsung	Galaxy S21, 128Go	5500.0 MAD	Modifier Supprimer
#4	Tablette iPad	Apple iPad Pro 11 pouces	9000.0 MAD	Modifier Supprimer
#5	Ecran LG 4K	LG 27 pouces 4K IPS	3200.0 MAD	Modifier Supprimer

- Interface principale : vue ADMIN avec liste des produits

Ajout d'un Produit

The screenshot shows the Admin interface with a green success message at the top: 'Produit ajouté avec succès'. The 'Ajouter un produit' form is visible on the left, and the 'Liste des produits' table is on the right.

Ajouter un produit

Form fields:

- Nom du produit *
- Description *
- Prix (MAD) *

Ajouter le produit

Liste des produits

Recherche par ID... Nom ou description... Chercher Reset

6 produit(s)

ID	NOM	DESCRIPTION	PRIX	ACTIONS
#1	PC Portable Dell	Dell Inspiron i7, 16Go RAM	8500.0 MAD	Modifier Supprimer
#2	PC Portable HP	HP Pavilion i5, 8Go RAM	6200.0 MAD	Modifier Supprimer

- Confirmation d'ajout : message de succès via RedirectAttributes

Recherche de Produits

Deux modes : recherche par **ID exact** (GET /produits/search?idProduit=X) et par **mot-clé** HQL LIKE (GET /produits/search?motCle=pc).

The screenshot shows the Admin interface with the search results for ID 2. The 'Liste des produits' table displays one product.

Liste des produits

Recherche par ID... Nom ou description... Chercher Reset

1 produit(s)

ID	NOM	DESCRIPTION	PRIX	ACTIONS
#2	PC Portable HP	HP Pavilion i5, 8Go RAM	6200.0 MAD	Modifier Supprimer

- Recherche par ID exact : GET /produits/search?idProduit=4

Recherche par ID...

2 produit(s)

ID	NOM	DESCRIPTION	PRIX	ACTIONS
#1	PC Portable Dell	Dell Inspiron i7, 16Go RAM	8500.0 MAD	Modifier Supprimer
#2	PC Portable HP	HP Pavilion i5, 8Go RAM	6200.0 MAD	Modifier Supprimer

- Recherche par mot-clé « pc » : 2 résultats

Modification d'un Produit

Modifier le produit

Nom du produit *

Description *

Prix (MAD) *

Liste des produits

Recherche par ID... Nom ou description...

6 produit(s)

ID	NOM	DESCRIPTION	PRIX
#1	PC Portable Dell	Dell Inspiron i7, 16Go RAM	8500.0 MAD
#2	PC Portable HP	HP Pavilion i5, 8Go RAM	6200.0 MAD
#3	Smartphone Samsung	Galaxy S21, 128Go	5500.0 MAD

Produit mis à jour

Ajouter un produit

Nom du produit *

Description *

Prix (MAD) *

Liste des produits

Recherche par ID...

6 produit(s)

ID	NOM
#1	PC Portable Dell
#2	PC Portable HP

- Formulaire pré-rempli : GET /produits/edit?id=X

Suppression et résultat

localhost:8081 says
Supprimer « sdd » ?

Ajouter un produit

Nom du produit *

Description *

Prix (MAD) *

Liste des produits

Recherche par ID... Nom ou description...

5 produit(s)

ID	NOM	DESCRIPTION	PRIX	ACTIONS
#1	PC Portable Dell	Dell Inspiron i7, 16Go RAM	6800.0 MAD	Modifier Supprimer
#2	PC Portable HP	HP Pavilion i5, 8Go RAM	6200.0 MAD	Modifier Supprimer
#3	Smartphone Samsung	Galaxy S21, 128Go	5500.0 MAD	Modifier Supprimer
#4	Tablette iPad	Apple iPad Pro 11 pouces	9000.0 MAD	Modifier Supprimer
#5	Ecran LG 4K	LG 27 pouces 4K IPS	3200.0 MAD	Modifier Supprimer

- Liste après suppression : POST /produits/delete?id=X

Vue Utilisateur : Profil USER

Avec le rôle USER : RoleFilter bloque l'accès aux URLs d'action (/produits/add, /edit, /update, /delete). Le formulaire CRUD est masqué par JSTL ().

Gestion des Produits USER

mode consultation

Les actions d'ajout, modification et suppression sont réservées aux ADMIN.

Liste des produits

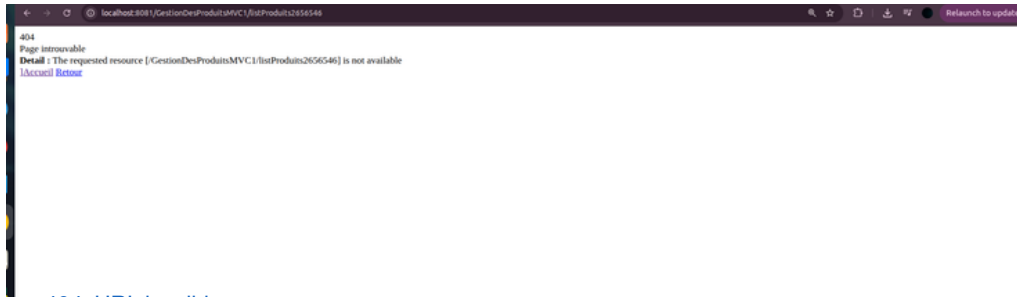
Recherche par ID... Nom ou description...

5 produit(s)

ID	NOM	DESCRIPTION	PRIX
#1	PC Portable Dell	Dell Inspiron i7, 16Go RAM	6800.0 MAD
#2	PC Portable HP	HP Pavilion i5, 8Go RAM	6200.0 MAD
#3	Smartphone Samsung	Galaxy S21, 128Go	5500.0 MAD
#4	Tablette iPad	Apple iPad Pro 11 pouces	9000.0 MAD
#5	Ecran LG 4K	LG 27 pouces 4K IPS	3200.0 MAD

- Interface en mode consultation : profil USER sans actions CRUD

Gestion des Erreurs HTTP



- Page d'erreur 404 :URL invalide

6

Conclusion

Ce TP a permis de maîtriser **Spring MVC**, le framework Java EE le plus utilisé en entreprise. Le passage de MVC2 manuel à Spring MVC apporte une **injection de dépendances automatique** (@Autowired), un **routing déclaratif** (@GetMapping, @PostMapping), et une meilleure **maintenabilité** du code.

Compétences acquises

- Configuration Spring MVC avec web.xml + spring-beans.xml + application-servlet-config.xml
- Injection de dépendances XML avec
- Annotations Spring MVC : @Controller, @Autowired, @GetMapping, @PostMapping, @RequestParam
- Persistance Hibernate + MySQL avec Session, Transaction, HQL
- Filtres de sécurité javax.servlet : AuthFilter + RoleFilter
- Pattern PRG (Post-Redirect-Get) avec RedirectAttributes
- ViewResolver : nom logique 'index' → /views/index.jsp

Élément	Détail
Projet	GestionDesProduits ; Maven Project Eclipse
Serveur	Apache Tomcat 9.0
Technologies	Spring MVC 5.3 + Hibernate 5.6 + JSTL + javax.servlet
Base de données	MySQL ;tables auto-générées par Hibernate (hbm2ddl.auto=update)
Architecture	Spring MVC + 3-tiers (DAO / Service / Controller)
Sécurité	AuthFilter (session) + RoleFilter (ADMIN) + Sessions HTTP
Rôles	ADMIN (CRUD complet) / USER (lecture seule)
Utilisateurs	admin/admin123 (ADMIN) -user/user123 (USER)
Résultat	Application web dynamique Spring MVC + Hibernate fonctionnelle